Actividad 1.3: Vulnerabilidades, lenguajes de bajo nivel.

Autor: Pedro Martín Escuela
Curso: Especialización Ciberseguridad

Índice

- 1. Desbordamiento de Búfer
- 2. Uso Tras Liberación
- 3. Doble Liberación
- 4. Cadena de Formato

Ejercicios

Para los siguientes ejemplos, encuentra la vulnerabilidad y explica cómo corregirla o mitigarla.

1. Desbordamiento de Búfer (Buffer Overflow)

Ocurre cuando se copian más datos en un búfer de los que este puede almacenar, sobrescribiendo la memoria contigua en la pila (stack).

```
// Código vulnerable

#include <stdio.h>
#include <string.h>

void funcion_vulnerable(char *input) {
    char buffer[16];
    strcpy(buffer, input);
    printf("Texto copiado: %s\n", buffer);
}

int main(int argc, char *argv[]) {
    if (argc > 1) {
        funcion_vulnerable(argv[1]);
    }
    return 0;
}
```

Vulnerabilidad Explicada:

En este ejemplo de código, se está permitiendo explotar la vulnerabilidad de **Buffer Overflow** mediante el uso de `strcpy(buffer, input)`. Aunque antes se haya especificado que `buffer` tiene un tamaño de **16**

caracteres, la función strcpy no comprueba el tamaño máximo de la variable, asi que si el input es mayor, ocurre un *overflow* sobrescribiendo asi la memoria contigua en la pila.

Además, un atacante puede diseñar el *input* de tal manera que reemplace la **dirección de retorno** (lo que va contiguo en la pila) con la dirección de memoria de un **código malicioso** (inyectado por él mismo), logrando ejecutar **comandos arbitrarios** en el sistema.

Solución:

Para solucionarlo, se debe cambiar por funciones que sí controlen los límites, como snprintf:

```
...
snprintf(buffer, sizeof(buffer), "%s", input);
...
```

2. Uso Tras Liberación (Use-After-Free - UAF)

Ocurre cuando un programa sigue utilizando un puntero después de que la memoria apuntada por este ha sido liberada con free().

```c
// Código vulnerable

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char nombre[20];
    void (*mostrar)(char *);
} Usuario;

void saludar(char *msg) {
    printf("Hola, %s\n", msg);
}

int main() {
    Usuario *usr = (Usuario *)malloc(sizeof(Usuario));
    strcpy(usr->nombre, "Alice");
    usr->mostrar = saludar;

    free(usr);

    printf("El usuario es: %s\n", usr->nombre);
    return 0;
}
```

Vulnerabilidad Explicada:

En este ejemplo de código, reserva espacio en memoria para guardar un valor en usr. Luego **marca como libre** la variable **usr**, pero no la libera automaticamente. Está marcando como liberada, es decir, el valor sigue estando asignado a la variable, solo le estás diciendo al compilador, que puede aprovechar ese espacio en memoria para sobreescribirlo y poner otra información.

El atacante puede interactuar con el programa (por ejemplo, enviando un comentario, cargando un archivo o creando otro objeto) que reserve exactamente el mismo tamaño de memoria.

Solución:

Para evitar un exploit **UAF**, lo mejor es limpiar siempre el puntero, asignándole NULL inmediatamente después de liberarlo.

```
...
    free(usr);
    usr=NULL;
...
```

3. Doble Liberación (Double Free)

Ocurre cuando se llama a free() dos veces sobre la misma dirección de memoria sin haber sido reasignada.

```c
// Código vulnerable

#include <stdio.h>
#include <stdlib.h>

void liberar_recurso(char *ptr) {
    free(ptr);
}

int main() {

    char *data = (char *)malloc(64);
    liberar_recurso(data);
    // ... lógica intermedia del programa …
    free(data);
    return 0;
}
```

Vulnerabilidad Explicada:

Si el programa realiza nuevos malloc más adelante, dos variables de diferentes partes del código podrían recibir exactamente la misma dirección de memoria. Esto permite que una parte del programa modifique sin querer (o de forma maliciosa) los datos confidentes o los punteros de función de otra estructura, derivando a menudo en un ataque de tipo Use-After-Free (UAF) o en la ejecución de código arbitrario.

Solución:

Prestar atención y no usar dos veces free sobre una misma variable. Recordemos que en el momento en que usa `free()` en una variable, solo marcamos como disponible el espacio en memoria, no la estamos liberando instantaneamente.

4. Cadena de Formato (Format String Vulnerability)

Ocurre cuando se pasa una entrada de usuario sin validar directamente como el primer argumento (format string) de funciones como printf sprintf o fprintf.

```
// Código vulnerable

#include <stdio.h>

void imprimir_mensaje(char *user_input) {
    printf(user_input);
    printf("\n");
}

int main(int argc, char *argv[]) {
    if (argc > 1) {
        imprimir_mensaje(argv[1]);
    }
    return 0;
}
```

Vulnerabilidad Explicada:

Al no validar lo introducido por el usuario, se puede usar argumentos que revelen informacón sensible existente en la pila o modifique la memoria.

Un ejemplo muy claro sería cambiar el valor de una variable como `is_admin = 0` -> `is_admin = 1` para que le de permisos al usuario que está intentando explotar la vulnerabilidad del programa.

Solución:

Siempre se debe dar un formato estático al argumento introducido por el usuario:

```
...
void imprimir_mensaje(char *user_input) {
    // El primer argumento SIEMPRE es una cadena fija (%s asegura que todo
lo demás sea tratado puramente como texto)
    printf("%s", user_input);
    printf("\n");
}
...
```